DOM APIs

Document Object Model

javaScript

ما هي DOM APIs ؟

DOM = Document Object Model

يعني ببساطة:
لما المتصفح يفتح صفحة HTML بيحولها لشجرة من العناصر (Objects) اسمها DOM.

اما DOM APIs هي الدوال والأوامر اللي بتسمح ل JavaScript تتعامل مع عناصر الصفحة.

يعني تقدر بـ JavaScript:

- **تغيير النص**
- **تغيير الألوان**
- **تضيف عناصر**
- **تمسح عناصر**
- **تتعامل مع الضغط على الأزرار**

أهم المفاهيم والدوال

1- Document
2- Node
3- Element
4- Attribute
5- Text Node
6- Parent Node
7- Child Node
8- Sibling Nodes
9- DOM Tree
10- DOM APIs

11- Selecting Elements
12- document.getElementById
13- document.getElementsByClassName
14- document.getElementsByTagName
15- document.querySelector
16- document.querySelectorAll

17- Traversing the DOM
18- parentNode
19- parentElement
20- childNodes
21- children
22- firstChild
23- firstElementChild
24- lastChild
25- lastElementChild
26- nextSibling
27- nextElementSibling
28- previousSibling
29- previousElementSibling

30- Manipulating Elements
31- innerHTML
32- textContent
33- style
34- classList
35- classList.add
36- classList.remove
37- classList.toggle
38- classList.contains

39- Attributes
40- getAttribute
41- setAttribute
42- removeAttribute
43- hasAttribute

44- Creating Elements
45- createElement
46- createTextNode
47- appendChild
48- append
49- prepend
50- before
51- after
52- insertAdjacentHTML
53- insertAdjacentElement
54- insertAdjacentText

55- Removing Elements
56- remove
57- removeChild

58- replaceChild
59- cloneNode

60- DOM Utilities
61- matches
62- closest
63- contains
64- dataset
65- nodeType
66- nodeName

67- Events
68- Event
69- Event Listener
70- addEventListener
71- removeEventListener
72- Event Bubbling
73- Event Capturing
74- Event Delegation
75- preventDefault
76- stopPropagation
77- DOMContentLoaded

1- Document

document ده كائن (Object) في JavaScript بيمثل كل صفحة الـ HTML.

ودة نقدر نقول مفتاح الوصول لباقي العناصر داخل الصفحة

```
Document.title="my new title";

//هنا غيرنا ال title بتاع الصفحة
```

2- Node

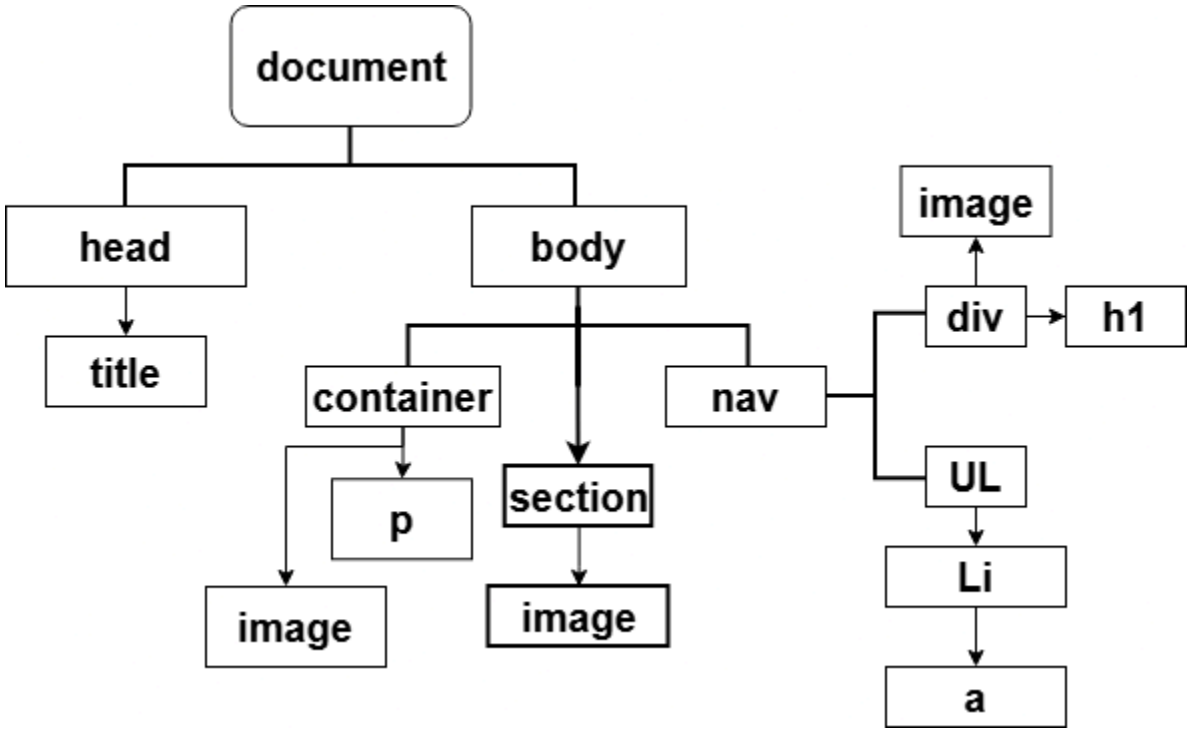
صفحة html عبارة عن شجرة كل عنصر فيها بيسمي Node
هنا document بيمثل node و body بيمثل node و head
بيمثل node الخ
الفكرة انك تعرف كل node في الـ project بتاعك جاية من اي
node وتعرف ال nodes الي طالعه منها
ودة بيسمي dom tree

3- DOM Tree

هي شجرة بتتكون من الـ nodes , بتبدء من ال document node

وتنتهي بآخر node في الصفحة

ودة رسم توضيحي



4-Element

هو اي وسم (tag) من وسوم html موجود في الصفحة
(h1 , p , div ,a) وغيرها

```
<h1>Hello</h1>
<p>Welcome</p>
```

h1	Element
p	Element

5- Attribute

هي خاصية تُكتب داخل الوسم الافتتاحي لعنصر HTML، وتُستخدم لإضافة معلومات أو إعدادات لهذا العنصر، وتكون بصيغة:

```
<a href="#">  </a>
```

هنا في open tag الخاص بالـ a ضيفنا

attribute	href
value	#

6- Text Node

هي أي كلمات وحروف مكتوبة بين وسمين

```
<p> i love DOM</p>
```

p	element node
I love DOM	text node

```
<button>click me</button>
```

button	element node
click me	text node

7- Parent Node

8- child Node

كلمة node قلنا إنها تمثل عنصر في الصفحة
parent node هو العنصر الذي يوجد بداخله عناصر
child node وهو العنصر الذي يوجد داخل ال parent node

```
<div>
    <p>Hello</p>
</div>
```

div	Parent node
p	Child node

9- Sibling Nodes

لما يكون عندك مجموعة عناصر لهم نفس الأب اسمه
sibling nodes يعني العناصر دول اخوة

```
<div>
    <h1>Title</h1>
    <p>Text</p>
    <button>Click</button>
</div>
```

كلهم داخل نفس العنصر الأب بتاعهم هو div

10- DOM APIs

هي مجموعة من الدوال التي توفرها javaScript للتعامل مع صفحة html ومن خلالها نقدر نعدل عناصر الصفحة وكمان نغير خصائص css للعناصر وكمان بيها ممكن ننشئ nodes جديدة أو attributes او نعدل على الموجود ونغير خصائص وامور تانيه كتير
يعني باختصار
"هي مجموعة دوال بتستخدم للتعامل مع صفحة ال html"

11- Selecting Elements

هي الطرق التي نحدد بها عناصر من صفحة HTML عشان نعدل عليها ب
javaScript فيما يلي سنتناول أهمها في الشرح

عند تحديد العناصر من html فإنه يفضل تخزينها في متغيرات

12- document.getElementById

وفي هذه الطريقة يتم استدعاء عنصر واحد من خلال ال id لأن الـ id Unique ولا يمكن تكراره فإذا كنت تريد تعديل عنصر محدد في الصفحة فيفضل اعطائة id واستدعائه من خلاله

```
let redParagraph = document.getElementById("redParagraph");
redParagraph.style.color="red";
redParagraph.style.fontWeight="bolder";
```

عندنا صفحة html فيها براجرافات اعطينا براجراف منهم id واستدعيناه في جافا سكريبت وخزناة في متغير ومن خلال المتغير عدلنا خواص css بكل سهولة دون الحاجة لاستدعاء العنصر من جديد في كل سطر

كمان نقدر نغير ونضيف attributes أو value للعناصر

```
let go =document.getElementById("go");
go.href = "https://www.wikipedia.org/";
```

هنا عندنا link له id اسمه go استدعيناه من خلال الـ id وعدلنا ال href بتاعه

13 - document.getElementsByClassName

ويمكننا ايضاً استدعاء العناصر بإستخدام ال class name
فهي تمكننا من استدعاء كل العناصر الذين يملكون نفس ال class
والتعديل عليهم جميعاً ببساطة
ولكن يرجى العلم ان التعديل علي اكثر من عنصر يملكون نفس ال
class لا يكون بتلك البساطة التي تتخيلها
عند اعطاء اكثر من عنصر نفس ال class فإن JS تتعامل معهم على
أنهم items فى Array ال Array هي الـ class فإن أردت أن
تتعامل مع جميع العناصر يجب عليك ان تقوم بعمل loop على
العناصر وكذلك يمكنك تحديد عنصر محدد باستخدام ال index

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <div id="container">
            <p class="odd">1</p>
            <p class="even">2</p>
            <p class="odd">3</p>
            <p class="even">4</p>
            <p class="odd">5</p>
            <p class="even">6</p>
            <p class="odd">7</p>
            <p class="even">8</p>
            <p class="odd">9</p>
    </div>
</body>
<script src="task.js"></script>
</html>
```

```
let container = document.getElementById("container");

container.style.display="flex";
container.style.backgroundColor="gray";
container.style.width="350px";
container.style.height="350px";
container.style.flexWrap ="wrap";
container.style.alignContent ="center";
container.style.alignItems ="center";
container.style.justifyContent ="center";

let odd = document.getElementsByClassName("odd");

for (let i =0 ; i < odd.length ; i++ ) {
    odd[i].style.width = "100px";
    odd[i].style.height = "100px";
    odd[i].style.margin = "0";
    odd[i].style.backgroundColor = "black";
    odd[i].style.color = "white";
    odd[i].style.display = "flex";
    odd[i].style.fontSize = "90px";
    odd[i].style.alignItems = "center";
    odd[i].style.justifyContent = "center";
}

let even = document.getElementsByClassName("even");

for (let i =0 ; i < even.length ; i++ ) {
    even[i].style.width = "100px";
    even[i].style.height = "100px";
    even[i].style.margin = "0";
    even[i].style.backgroundColor = "white";
    even[i].style.color = "black";
    even[i].style.display = "flex";
    even[i].style.fontSize = "90px";
    even[i].style.alignItems = "center";
    even[i].style.justifyContent = "center";
}
```

14- querySelector()

هي دالة تستخدم لاختيار عنصر واحد من صفحة html باستخدام محدد css

```
let paragraph =
document.querySelector("#two");
paragraph.style.color = "red";
```

```
<p class="odd">1</p>
<p class="even" id="two">2</p>
<p class="odd">3</p>
```

15- document.querySelectorAll()

هي دالة تستخدم لتحديد مجموعة عناصر تطابق محدد **css** معين وترجع **nodeList**

```
let paragraph = document.querySelectorAll(".num");
    for(i = 0 ; i < paragraph.length ; i++){
        paragraph[i].style.color = "blue";
    }
```

```
<p class="num">1</p>
<p class="num">2</p>
<p class="num">3</p>
```

16- Traversing the DOM

هي عملية التنقل بين عناصر شجرة الـ DOM، حيث تُمكِّنك بعد تحديد عنصر معيّن من الوصول إلى العناصر المرتبطة به مثل **العنصر الأب (Parent)** أو **الأبناء (Children)** أو **العناصر الشقيقة (Siblings)** باستخدام مجموعة من الدوال في **JavaScript**.

18- parentNode

تستخدم للوصول لل Node الأب للعنصر الحالي

هنا عندنا div جواة مجموعة من البراجرافات اعطينا براجراف منهم id	`<div>`
	`  <p> i love css</p>`
	`  <p id="paragraph"> i love DOM</p>`
	`  <p> i love javaScript</p>`
	`</div>`

في كود js عملنا متغير و استدعينا فيه البراجراف باستخدام الـ id بعد كدة قمنا جايبين المتغير الي فيه البراجراف واستعملنا parentNode للوصول للأب الي هو الـ div وغيرنا لونة للون الأحمر	`let paragraph = document.querySelector("#paragraph");` `paragraph.parentNode.style.background ="red";`

19- parentElement

تختلف أي عن parentNode ؟

المقارنة	parentNode	parentElement
التعريف	خاصية تعيد العقدة الأب للعنصر	خاصية تعيد العنصر الأب فقط
نوع القيمة المعادة	يمكن أن تكون أي Node	تكون Element فقط
ما الذي يمكن أن تعيده	Element Node أو Document أو أنواع Nodes أخرى	عنصر HTML فقط
إذا كان الأب ليس Element	يعيده عادي	تعيد null
لاستخدام	عند التعامل مع كل أنواع الـ Nodes	عند التعامل مع عناصر HTML فقط

20- childNodes

دي بترجعلك كل ال Nodes الي جوة العنصر على سبيل المثال كود زي دة

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
        <div id="container">
            <p> i love css</p>
            <p> i love DOM</p>
            <p> i love javaScript</p>
        </div>
    <script src="task.js"></script>
</body>
</html>
```

```
let paragraph = document.querySelector("#container").childNodes;
console.log(paragraph);
```

قد تظن للوهلة الأولى أنه هيطبع البراجرافات بس

الا انه في الحقيقه هيطلع كل الـ Nodes بالشكل دة

```
▼ NodeList(7) [text, p, text, p, text, p, text] i
  ▶ 0: text
  ▶ 1: p
  ▶ 2: text
  ▶ 3: p
  ▶ 4: text
  ▶ 5: p
  ▶ 6: text
    length: 7
  ▶ [[Prototype]]: NodeList
```

طب دلوقتي انا عايز اغير لون البراجرافات اخليه احمر هعمل كدة ازاي باستخدام childNodes

الحل:

```
let paragraph = document.querySelector("#container").childNodes;
console.log(paragraph);

for (i = 0; i < paragraph.length; i++) {
    if(paragraph[i].nodeName === "P"){
        paragraph[i].style.color = "red";
    }
}
```

21- Children

Children بترجع ال Elements فقط علي عكس childNodes
الي كانت بترجع كل الـ nodes سواء elements او غيرهم فبالتالي
Children هي الأفضل عند التعامل مع الـ Elements داخل
العنصر
يلا نجرب المثال السابق ولكن باستخدام children

```
let paragraph = document.querySelector("#container").children;

console.log(paragraph);
for (i = 0; i < paragraph.length; i++) {

    paragraph[i].style.color = "red";

}
```

22- firstChild

دي بتجيب اول Node جوة العنصر سواء Element او Text او اي Node

23- lastChild

دي بتجيب اخر Node جوة العنصر سواء Element او Text او اي Node

24- firstElementChild

دي بتجيب اول Element جوة العنصر

25- lastElementChild

دي بتجيب اخر Element جوة العنصر

26- nextSibling

بتجيب الأخ التالي للعنصر في نفس الأب وبتجيب اي Node يعني مش Element بس ولذلك لا يفضل استعمالها عند التعامل مع Elements

27- previousSibling

بتجيب الأخ السابق للعنصر في نفس الأب وبتجيب اي Node يعني مش Element بس ولذلك لا يفضل استعمالها عند التعامل مع Elements

28- nextElementSibling

بتجيب الأخ المباشر للعنصر المشترك معاه في نفس الأب وبتجيب Element بس

```
let paragraph = document.querySelector("#paragraph").nextElementSibling;
    paragraph.style.color = "red";
```

29- previousElementSibling

بتجيب الأخ السابق للعنصر المشترك معاه في نفس الأب وبتجيب Element بس

```
let paragraph = document.querySelector("#paragraph").previousElementSibling;
    paragraph.style.color = "red";
```

30- Manipulating Elements

التعامل مع العناصر وتعديلها

يعني:

إنت تمسك عنصر في الصفحة (زي `div` أو `p` أو `button`)

وبعدين تغير فيه أو تضيفله حاجة أو تمسحه أو تعدل شكله أو محتواه بالجافاسكربت

31- innerHTML

1- بتغير الي جوة العنصر اللي احنا محددينه قبلها
يعني بتستبدل محتوى العنصر تماماً

```
<div id="container">
    <p id="paragraph"> i love css</p>
    <p> i love DOM</p>
    <p> i love javaScript</p>
</div>
```

```
let paragraph = document.querySelector("#container");
    paragraph.innerHTML = "red";
```

الناتج هيستبدل كل البراجرافات بكلمة red

2- كمان تقدر من خلالها تضيف elements بمعنى اكتب فيها اكواد html وهتتضاف للصفحة كعناصر html مش مجرد text

```
let paragraph = document.querySelector("#container");
paragraph.innerHTML = "<div> <p> hello world </p> </div>";
```

دة هيضيف جوة ال div الي احنا استدعيناه div جواة براجراف جوة البراجراف hello world بالتالي اللي هيظهر في الصفحة
hello world بس
وليس
** hello world **
السطر ده هيعتبرة كود html

3- اذا عايز تضيف عالي في العنصر دون مسح محتواه اسعمل +=

```
let paragraph = document.querySelector("#container");
paragraph.innerHTML += "<div> <p> hello world </p> </div>";
```

بكدة الناتج هيكون

i love css

i love DOM

i love javaScript

hello world

إمتى أتجنب `innerHTML`؟

1- لو البيانات جاية من المستخدم مباشرة لأن ده ممكن يعمل مشاكل أمنية اسمها: XSS Cross-Site Scripting

2- لو عايز تضيف عناصر كتير بشكل احترافي

3- لو خايف على الأمان

32- textContent

ممكن تستعملها في :
تجيب النص اللي جوة العنصر
إذا حاولت اضافة عناصر html زي ما عملنا في innerHTML
بتضيفها كنص مش هتعتبرها html

```
<div id="box">
   <p>apple</p>
   <p>tree</p>
</div>
```

```
let box = document.getElementById("box");
console.log(box.textContent);
```

output	**apple** **tree**

تعديل النص اللي جوة العنص

```
<div id="box"></div>
```

```
let box = document.getElementById("box");
box.textContent = "hello world";
```

output	**hello world**

33- style

ملاحظة هامة في css بنكتب الأوامر بالشكل دة
background-color في جافا سكريبت بنكتب بطريقة
camelCase فبتكون backgroundColor

```
document.body.style.backgroundColor = "#001D39";
ونلاحظ هنا كمان ال value مكتوبة بين علامتين تنصيص
```

خلينا نشرح الكود دة تفصيلياً
document : هو الكائن الي بيمثل الصفحة
dot (.) : كل ما نتعمق أكثر داخل العناصر بنستخدمها
style : للعنصر المختار وما بعدها خاصية css لتنسيق

الموضوع ببساطة ان بنكتب style وبعدها الدوت ثم خاصية css
بين علامتين تنصين value وبنعمل يساوي وبعد كدة بنعمل ال

34- classList

لما يكون عندي عنصر html زي كدة

```
<div id="box" class="card active"></div>
```

فكل class بديها للعنصر بتخزن في حاجة بنسميها class list
فلو جربنا بجافا سكريبت

```
let box = document.getElementById("box");

console.log(box.classList);
```

هيطلعلك list فيها card active

35- classList.add

دي بتضيف class جديدة لل classList
يعني لو عندك list ليها Class وعايز تضيف ليها كمان بنستعمل
classList.add ولو عندك عنصر ملوش اي class بنستعملها

```
element.classList.add("className");
```

وممكن تضيف أكثر من class ببساطة بالطريقة دي

```
text.classList.add("red", "big");
```

36- classList.remove

نفس اسلوب استخدام add بس هنا يحذف class او اكثر على عكس add

37- classList.toggle

لو ال class موجودة في class List بتاعت العنصر يشيلها ولو مش موجودة يضيفها
وبتاخد class واحدة بس علي عكس add و remove

```
box.classList.toggle("ClassName");
```

38- classList.contains

بنستخدمها عشان نعرف اذا كان ضمن الـ classList بتاعة العنصر Class معين ولا لا

```
element.classList.contains("className");
```

بترجع true او false

39- getAttribute

بيجيب قيمة attribute معينة من العنصر

قولنا قبل كده ان ال attributes هي خواص بتتضاف لل open tag
بتاع العنصر لاضافة او تغيير خصائص العنصر

html

```
<a href="https://google.com" target="_blank">Google</a>
```

الصيغة:

```
element.getAttribute("اسم الخاصية")
```

```
element.getAttribute("href")
```

output:

```
https://google.com
```

لو ال Attribute مش موجودة هيرجعلك null

40- setAttribute

بتقوم بإضافة attribute او تعديل Attribute موجودة مسبقاً

```
element.setAttribute("Attribute", "Value")
```

تعديل Attribute موجود بالفعل

HTML

```
<img src="old.jpg">
```

JavaScript

```
let img = document.querySelector("img");
img.setAttribute("src", "new.jpg");
```

إضافة Attribute جديد

HTML

```
<img src="image.jpg">
```

JavaScript

```
img.setAttribute("alt", "new img");
```

41- removeAttribute

وظيفتة بيحذف Attribute

الصيغة:

```
element.removeAttribute("اسم الخاصية")
```

42- hasAttribute

وظيفته ايه؟

بيتأكد إذا كان العنصر عنده الـ attribute دي ولا لا.

الصيغة:

```
element.hasAttribute("اسم الخاصية")
```

بيرجع true او false

43- createElement

الصيغة :

```
document.createElement("tagName")
```

بتعمل عنصر جديد زي

div

a

p

```
let div = document.createElement("div");
```

كدة ال div اتعمل بس لسه مش ظاهر في الصفحة

مثال كامل لإنشاء وإضافة عنصر للصفحة:

```
let p = document.createElement("p");
p.textContent = "Hello From JavaScript";
document.body.appendChild(p);
```

44- createTextNode

بتضيف نص للصفحة (text node)
مش بتضيف p مش بتضيف div هي بتضيف مجرد نص

مثال:

```
let p = document.createElement("p");
let text = document.createTextNode("Hello From
JavaScript");
p.appendChild(text);
document.body.appendChild(p);
```

هنا عملنا براجراف
بعدين عملنا textNode
بعدين ضفنا ال text جوة البراجراف
بعدين ضفنا ال p لل body

45- appendChild

تقوم بإضافة عنصر داخل عنصر العنصر المضاف يتم وضعه كأخر عنصر داخل العنصر الأب

مثال :

```
document.body.appendChild(div);
```

46- append

بتستخدم عشان تضيف عنصر أو أكثر داخل عنصر
بتضيف العناصر في اخر العنصر الأب
مثال :

```
let p = document.createElement("p");
p.textContent = "Hello";

document.body.append(p);
```

الفرق بين append و append child

append

ممكن يضيف عنصر أو أكثر وممكن يضيف نص

appendChild

بيضيف عنصر واحد بس ومش بيقبل نص مباشر

مثال لإضافة أكثر من عنصر باستخدام append

```
let box = document.getElementById("box");

let h1 = document.createElement("h1");
h1.textContent = "Title";

let p = document.createElement("p");
p.textContent = "Description";

box.append(h1, p);
```

47- prepend

بتضيف العناصر في بداية العنصر الأب على عكس append اللي بتضيف في الأخر

```
let container = document.querySelector("#container");
let paragraph = document.createElement("p");
let text = document.createTextNode("from append");
let paragraph2 = document.createElement("p");
let text2 = document.createTextNode("from prepend");
paragraph.append(text);
container.append(paragraph);
paragraph2.prepend(text2);
container.prepend(paragraph2);
```

الناتج :

from prepend

i love css

i love DOM

i love javaScript

from append

48- before

بتضيف عنصر أو نص قبل العنصر الأب يعني بتضيف العنصر أو النص قبل العنصر المحدد قبلها مش جواه

الصيغة :

```
element.before(item1, item2, "text");
```

يقبل إيه؟

1) عنصر واحد

```
element.before(div);
```

2) أكثر من عنصر

```
element.before(h1, p);
```

3) نص عادي String

```
element.before("Hello");
```

4) نص + عناصر

```
element.before("Start ", strong);
```

49- after

هي نفس before بس بينهم فرق بسيط before بتضيف عناصر قبل العنصر after بتضيف بعد العنصر

50- insertAdjacentHTML

لكي نتعامل مع عناصر من الصفحة علينا تعلم استدعائها ويفضل عند استدعائها تخزينها داخل متغير لنتمكن من إعادة التعامل معه بسهولة

يفضل جعل تاج جافا سكريبت في نهاية صفحة ال html
و هذه هي صفحة ال html التي سنتعامل معها خلال الشرح

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <link rel="stylesheet" href="big.css">
    <title>Document</title>
</head>

<body>
    <h1>Hello, World!</h1>
    <p>This is a simple HTML document.</p>
```

```html
    <p>This paragraph is empty.</p>
    <span class="big-text">This text is big!</span>
    <div class="big-box">This box is big!</div>
    <div id="my-id">This div has an ID.</div>
    <form action="">
        <input type="text" name="one" value="hello">
    </form>
    <form action="">
        <input type="text" name="two" value="hello">
    </form>
    <a href="https://google.com">Google</a>
    <a href="https://facebook.com">Yahoo</a>
    <script src="big.js"></script>


</html>
```

طرق تحديد عناصر html

ويمكننا ايضاً استدعاء العناصر بإستخدام ال tag
تمكننا من استدعاء كل العناصر الذين يملكون نفس الـ tag والتعديل عليهم جميعاً ببساطة
ولكن يرجى العلم ان التعديل علي اكثر من عنصر يملكون نفس ال tag لا يكون بتلك البساطة الذي تتخيلها

عند اعطاء اكثر من عنصر نفس الـ tag فإن JS تتعامل معهم علي أنهم items فى Array ال Array هي الـ array فإن أردت أن تتعامل مع جميع العناصر يجب علي ان تقوم بعمل loop علي العناصر

```
<p class="odd">1</p>
<p class="even">2</p>
<p class="odd">3</p>
<p class="even">4</p>
<p class="odd">5</p>
<p class="even">6</p>
<p class="odd">7</p>
<p class="even">8</p>
<p class="odd">9</p>
```

```
let paragraph = document.getElementsByTagName("p");
for (let i =0 ; i < paragraph.length ; i++ ) {
    paragraph[i].style.color = "red";
}
```

أو تحديد احد العناصر باستخدام ال index

```
var myparagraphs = document.getElementsByTagName("p");
console.log(myparagraphs[0]);
```

```
/*
  DOM [Class List]
  - classList
  --- length
  --- contains
  --- item(index)
  --- add
  --- remove
  --- toggle
*/

let element = document.getElementById("my-div");

console.log(element.classList);
console.log(typeof element.classList);
console.log(element.classList.contains("osama"));
console.log(element.classList.contains("show"));
console.log(element.classList.item("3"));

element.onclick = function () {
  element.classList.toggle("show");
};
```

التعديل في الصفحة

كود html الي سيتم التعامل معاة في الامثلة التالية

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <link rel="stylesheet" href="big.css">
```

```
    <title>Document</title>
</head>

<body>
    <div class="js"> java script <span>div</span></div>
    <img src="" alt="">
    <a href="#">go</a>
    <script src="big.js"></script>
,ظلاخيغ.

</html>
```

3-document.images

تقوم بالتعديل علي الصور

```
var myelement = document.querySelector(".js");
myelement.textContent = "<span>hello world</span>";
// <span>hello world</span>
document.images[0].src = "https://blablabla";
//تعديل المسار
document.images[0].alt = "Google Logo";
//تعديل النص البديل
document.images[0].title = "Google";
//تعديل العنوان
document.id = "myimage";
```

```
//تعديل الايدي
document.images[0].className = "image";
//تعديل الكلاس
```

4-document.link

تقوم بالتعديل علي الروابط

```
let mylink = document.querySelector('.link');

mylink.setAttribute('href', 'https://google.com');
// هذا يغير الرابط إلى
// google
mylink.setAttribute('title', 'Google');
// هذا يغير عنوان الرابط إلى
// Google
mylink.textContent = 'Google';
// هذا يغير كلمة
// go
//  إلى
// Google
```

معرفة معلومات عن العنصر

1-.attributes

تظهر جميع ال attributes الخاصة بالعنصر

```
console.log(document.getElementsByTagName("p")[0].attributes);
```

2-.hasAttribute

- الوظيفة: تتحقق مما إذا كان العنصر يحتوي على سمة معينة
- إذا كانت السمة موجودة `true`
- إذا لم تكن موجودة `false`

مثال:

```
let el = document.getElementById("myDiv");
el.hasAttribute("class");
 // true
 // إذا كان للعنصر سمة
 //class
```

3-.hasAttributes

تتحقق مما إذا كان العنصر يحتوي على أي سمات على الإطلاق

اذا كان للعنصر أي سمة النتيجة => true

اذا لم يكن للعنصر أي سمة النتيجة => false

```
let el = document.getElementById("myDiv");
el.hasAttributes();
```

مثال اكبر:

```
/*
  DOM [Check Attributes]
  - Element.attributes
  - Element.hasAttribute
  - Element.hasAttributes
  - Element.removeAttribute
*/
```

```
console.log(document.getElementsByTagName("p")[0].attributes);

let myP = document.getElementsByTagName("p")[0];

if (myP.hasAttribute("data-src")) {
  if (myP.getAttribute("data-src") === "") {
    myP.removeAttribute("data-src");
  } else {
    myP.setAttribute("data-src", "New Value");
  }
} else {
  console.log(`Not Found`);
}

if (myP.hasAttributes()) {
  console.log(`Has Attributes`);
}

if (document.getElementsByTagName("div")[0].hasAttributes()) {
  console.log(`Has Attributes`);
} else {
  console.log(`Div Has No Attributes`);
}
```

Create And Append

```
let myElement = document.createElement("div");
//createElement
let myAttr = document.createAttribute("data-custom");
// createAttribute
let myText = document.createTextNode("Product One");
```

```
// createTextNode
let myComment = document.createComment("This Is Div");
// createComment
myElement.className = "product";
// set class name
myElement.setAttributeNode(myAttr);
// setAttributeNode
myElement.setAttribute("class", "box");
// خلي العنصر اسمة
// DATA-TEST
// وضع له القيمة
// Testing
myElement.appendChild(myComment);
// Append Comment To Element
myElement.appendChild(myText);
// Append Text To Element
document.body.appendChild(myElement);
// Append Element To Body
```

التعامل مع الابناء

```
let myElement = document.querySelector("div");

console.log(myElement);
console.log(myElement.children);
// يرجع من العناصر  الابناء جميع العناصر
console.log(myElement.children[0]);
// يرجع من العناصر  الابناء العنصر الاول فقط
console.log(myElement.childNodes);
// يرجع من العناصر  الابناء جميع العناصر و النصوص
console.log(myElement.childNodes[0]);
// يرجع من العناصر  الابناء اول عنصر او نص
console.log(myElement.firstChild);
console.log(myElement.lastChild);
console.log(myElement.firstElementChild);
console.log(myElement.lastElementChild);
```

Events

الأحداث

الأحداث وهي انه عند حدوث حدث معين ينفذ كود معين وهي مهمة جداً جداً وايضاً هي كثيرة فما سنوضحة هنا بعض الأمثلة الهامة وللإطلاع عليها جميعاً تصفح الويب

الأحداث هي الطريقة اللي بتخلي صفحة الويب سايت يتفاعل مع المستخدم. لما المستخدم يعمل حاجة زي النقر أو الكتابة أو التمرير، تقدر ترد عليه بكود معين.

🖱️ أحداث الماوس

الوصف	الحدث
عند النقر على العنصر	click
عند النقر مرتين بسرعة (نقرة مزدوجة)	dblclick
عند مرور الماوس فوق العنصر	mouseover
عند خروج الماوس من العنصر	mouseout
عند تحريك الماوس داخل العنصر	mousemove

click - 1

```
let changeBackgrounColor = document.getElementById("changeBackgrounColor");
changeBackgrounColor.addEventListener("click", () => {
    if (document.body.style.backgroundColor == "blue") {
        document.body.style.backgroundColor = "black";
    }
    else{
        document.body.style.backgroundColor = "blue";
    }
})

// في الكود هنا عندنا زرار في html استدعيناه باستخدام الـ id وحطيناه في
متغير بعدين استعملنا addEventListener اول حاجة اعطيناها
```

```
click attribute
وده معناه عند النقر على العنصر الي انا محددة
القيمة الثانية وهي function جواها code block بيغير لون خلفية الموقع
بمجرد النقر على الزرار ال code block هيتنفذ
```

مثال:

```
button.addEventListener("click", function() {
  alert("تم النقر!");
});
```

⌨️ **أحداث لوحة المفاتيح**

الحدث	الوصف
keydown	عند الضغط على زر في الكيبورد
keyup	عند رفع الإصبع عن الزر
keypress	عند الضغط المستمر على الزر

مثال:

```
dobutton.addEventListener("click", function() {
  alert("تم النقر!");
});function(e) {
  console.log("ضغطت على: " + e.key);
```

```
});
```

أحداث النموذج (Forms)

الوصف	الحدث
عند إرسال النموذج	submit

عند تغيير قيمة عنصر مثل `<input>` أو `<select>`	change
`focus`	عند التركيز على عنصر الإدخال
`blur`	عند فقدان التركيز من عنصر الإدخال

مثال:

```
form.addEventListener("submit", function(e) {
  e.preventDefault(); // يمنع الإرسال الفعلي
  alert("تم إرسال النموذج!");
});
```

🌐 أحداث الصفحة

الوصف	الحدث
عند اكتمال تحميل الصفحة أو العنصر	load
عند تغيير حجم نافذة المتصفح	resize
عند تمرير الصفحة لأعلى أو لأسفل	scroll

مثال:

```
window.addEventListener("load", function() {
  console.log("الصفحة جاهزة!");
});
```

التحقق من بيانات المستخدم قبل الارسال

```
let userInput = document.querySelector("[name='username']");
let ageInput = document.querySelector("[name='age']");

document.forms[0].onsubmit = function (e) {
    let userValid = false;
    let ageValid = false;
    if (userInput.value !== "" && userInput.value.length <= 10) {
        userValid = true;
    }
    //التحقق من صحة اسم المستخدم
    if (ageInput.value !== "") {
        ageValid = true;
    //التحقق من صحة العمر
    }
    if (userValid === false || ageValid === false) {
        e.preventDefault();
    }
};
document.links[0].onclick = function (event) {
    console.log(event);
    event.preventDefault();
};
```

Event Simulation - Click Focus Blur

```
let one = document.querySelector(".one");
let two = document.querySelector(".two");

window.onload = function () {
  two.focus();
};

one.onblur = function () {
  document.links[0].click();
};
```

Before, After, Prepend, Append, Remove

```
/*
  DOM [Deal With Elements]
  - before [Element || String]
  - after [Element || String]
  - append [Element || String]
  - prepend [Element || String]
  - remove
*/

let element = document.getElementById("my-div");
let createdP = document.createElement("p");

// element.remove();
```

DOM Traversing

```
/*
  DOM [Traversing]
  - nextSibling
  - previousSibling
  - nextElementSibling
  - previousElementSibling
  - parentElement
*/

let span = document.querySelector(".two");

console.log(span.parentElement);

span.onclick = function () {
  span.parentElement.remove();
}
```

Cloning

AddEventListener